Day-24

Python 90 Days Challenges

Pillars of OOP

Encapsulation • Inheritance • Polymorphism • Abstraction

Point	Instance Variable	Class Variable
Belongs to	Object	Class
Created where	Usually inside __init__	Inside class, outside methods
Shared?	No	Yes
Access	self.name	ClassName.variable
Example	self.name	school_name

Important Warning

Avoid mutable class variables unless shared data is intentional.

```python
class Student:
    subjects = []
```

This list is shared by all objects, which can cause unexpected behavior.

14.1.9 Methods vs Functions

A function is independent.
A method belongs to a class or object.

Comparison Table

Point	Function	Method
Defined where?	Outside class	Inside class
Called using	Function name	Object or class
First parameter	No automatic self	Instance method gets self
Example	greet()	student.greet()

14.2 OOPS Principles

14.2.1 Encapsulation

Code With Nishchal

Encapsulation means **binding data and methods together inside a class** and controlling how data is accessed or modified.

Basic Idea

Data + Methods = Encapsulation

1. Encapsulation keeps related data and behavior inside one class.
2. It helps protect data from direct unwanted changes.
3. Python does not have strict private variables like Java or C++.
4. Python uses naming conventions to show access level.
5. Encapsulation is commonly handled using:
    1. Public attributes
    2. Protected attributes
    3. Private/name-mangled attributes
    4. Getter and setter methods
    5. @property

PYTHON CODE

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
account.deposit(500)
print(account.get_balance())
```

Output: 1500

Here, __balance is not directly accessed from outside. It is accessed using methods.

Public, Protected, and Private Members

Python uses naming conventions for access control.

Access Level Table

Nishchal

Type	Syntax	Meaning	Example
Public	name	Can be accessed normally	self.name
Protected	_name	Meant for internal use	self._salary
Private / Name mangling	__name	Avoid accidental outside access	self.__pin

Example 1

```
PYTHON CODE
class Student:
    def __init__(self):
        self.name = "Aman"
        self._marks = 85
        self.__grade = "A"

student = Student()
print(student.name)
print(student._marks)
```

Output: Aman
85
Direct access to __grade will fail:
print(student.__grade)
Output: AttributeError: 'Student' object has no attribute '__grade'

Important Point

Double underscore does not make data fully private.
It performs name mangling to avoid accidental access.
Internally, Python changes:
__grade -> _Student__grade
So technically it can still be accessed, but it should not be used directly.

Getter and Setter Methods

Getter and setter methods are used to read and update private data safely.

CodeWithNishchal

```python
def get_value(self):
    return self.__value

def set_value(self, value):
    self.__value = value
```

1. Getter method reads private data.
2. Setter method updates private data.
3. Setter can validate data before updating.
4. This protects the object from invalid values.

Example 1

PYTHON CODE

```python
class Student:
    def __init__(self, marks):
        self.__marks = marks

    def get_marks(self):
        return self.__marks

    def set_marks(self, marks):
        if marks >= 0:
            self.__marks = marks
        else:
            print("Marks cannot be negative")
student = Student(80)
student.set_marks(90)
print(student.get_marks())
```

Output: 90

Encapsulation Using @property

Nishchal

@property allows a method to behave like an attribute.

Syntax

```python
@property
def attribute_name(self):
    return value
```

1. @property is a clean way to control access to data.
2. It allows validation before changing data.
3. It supports getter and setter behavior.
4. It makes code look simple while still protecting data.

Example 1

PYTHON CODE

```
class Student:
    def __init__(self, marks):
        self.marks = marks

    @property
    def marks(self):
        return self._marks

    @marks.setter
    def marks(self, value):
        if value < 0:
            raise ValueError("Marks cannot be
negative")
        self._marks = value

student = Student(85)
student.marks = 95
print(student.marks)
```

Output: 95

14.2.2 Inheritance

Nishchal

Inheritance allows one class to reuse the properties and methods of another class.

Syntax

```python
class ChildClass(ParentClass):
    statement
```

Explanation

1. Inheritance supports code reuse.
2. The parent class is also called base class or superclass.
3. The child class is also called derived class or subclass.
4. The child class can use parent class attributes and methods.
5. The child class can also define its own attributes and methods.
6. The child class can override parent methods.
7. Inheritance represents an **is-a** relationship.

Basic Example

PYTHON CODE

```python
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
        print("Barking")
dog = Dog()
dog.eat()
dog.bark()
```

Output: Eating
Barking

Inheritance Flow

Parent class

hNishchal

Overriding with super()

```
PYTHON CODE
class Animal:
    def sound(self):
        print("Animal sound"
class Dog(Animal):
    def sound(self):
        super().sound()
        print("Bark")
dog = Dog()
dog.sound()
```

Output: Animal sound
Bark

14.2.4 Polymorphism

Polymorphism means the same method or operation behaves differently for different objects.

Explanation

1. Polymorphism means “many forms”.
2. The same method name can work differently in different classes.
3. Python supports polymorphism through:
    1. **Method overriding**
    2. **Duck typing**
    3. **Operator overloading**

```
PYTHON CODE
class Dog:
    def sound(self):
        print("Bark")


class Cat:
    def sound(self):
        print("Meow")
animals = [Dog(), Cat()]
for animal in animals:
    animal.sound()
```

Output: Bark

Meow

WithNishchal

Polymorphism Types in Python

Type	Meaning	Example
Method overriding	Child changes parent method	Dog.sound()
Duck typing	Object is accepted if it has required method	animal.sound()
Operator overloading	Operator works differently by class	obj1 + obj2

14.2.5 Duck Typing

Duck typing means Python focuses on what an object can do, not its exact type.

Basic Idea

If an object has the needed method, Python can use it.

Example 1

PYTHON CODE

```
class Duck:
    def speak(self):
        print("Quack")

class Person:
    def speak(self):
        print("Hello")

def call_speak(obj):
    obj.speak()

call_speak(Duck())
call_speak(Person())
```

Output:
Quack
Hello

Python does not care whether the object is Duck or Person. It only checks whether speak() exists.

Nishchal

14.2.6 Abstraction

Abstraction means hiding internal implementation and showing only essential features.

Explanation

1. Abstraction focuses on what an object does.
2. It hides how the object does it internally.
3. Python supports abstraction using abstract base classes.
4. Abstract base classes are created using the abc module.
5. A class with abstract methods cannot be instantiated directly.
6. Child classes must implement abstract methods.

Example 1

PYTHON CODE

```
from abc import ABC, abstractmethod

class Payment(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class UpiPayment(Payment):
    def pay(self, amount):
        print("Paid", amount, "using UPI")

payment = UpiPayment()

payment.pay(500)
```

Output: Paid 500 using UPI

Important Point

Payment cannot be used directly because it has an abstract method.

This gives an error:

payment = Payment()

Output:

TypeError: Can't instantiate abstract class Payment with abstract method pay

Nishchal

Encapsulation vs Abstraction

Point	Encapsulation	Abstraction
Meaning	Protecting and controlling data	Hiding implementation details
Focus	Data access	Essential behavior
Achieved by	Private variables, methods, @property	Abstract classes, interfaces
Example	Hide __balance	Define pay() without showing payment logic

Encapsulation = How data is protected
Abstraction = How unnecessary details are hidden

14.2.7 Composition

Composition means one class contains an object of another class.

Basic Idea

Inheritance = is-a relationship
Composition = has-a relationship

1. Composition is used when one object is made of another object.
2. It represents a **has-a** relationship.
3. It is often preferred over inheritance when there is no true is-a relationship.

Example 1

```
PYTHON CODE
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()
car = Car()
car.start()
```

Output:Engine started
Explanation:
Car has an Engine.
So this is composition.

WithNishchal